NICS 清華大學

Tsinghua University

Modern Computer Architecture
Fall 2025

Lecture 08_2: Virtual Memory Hardware Support

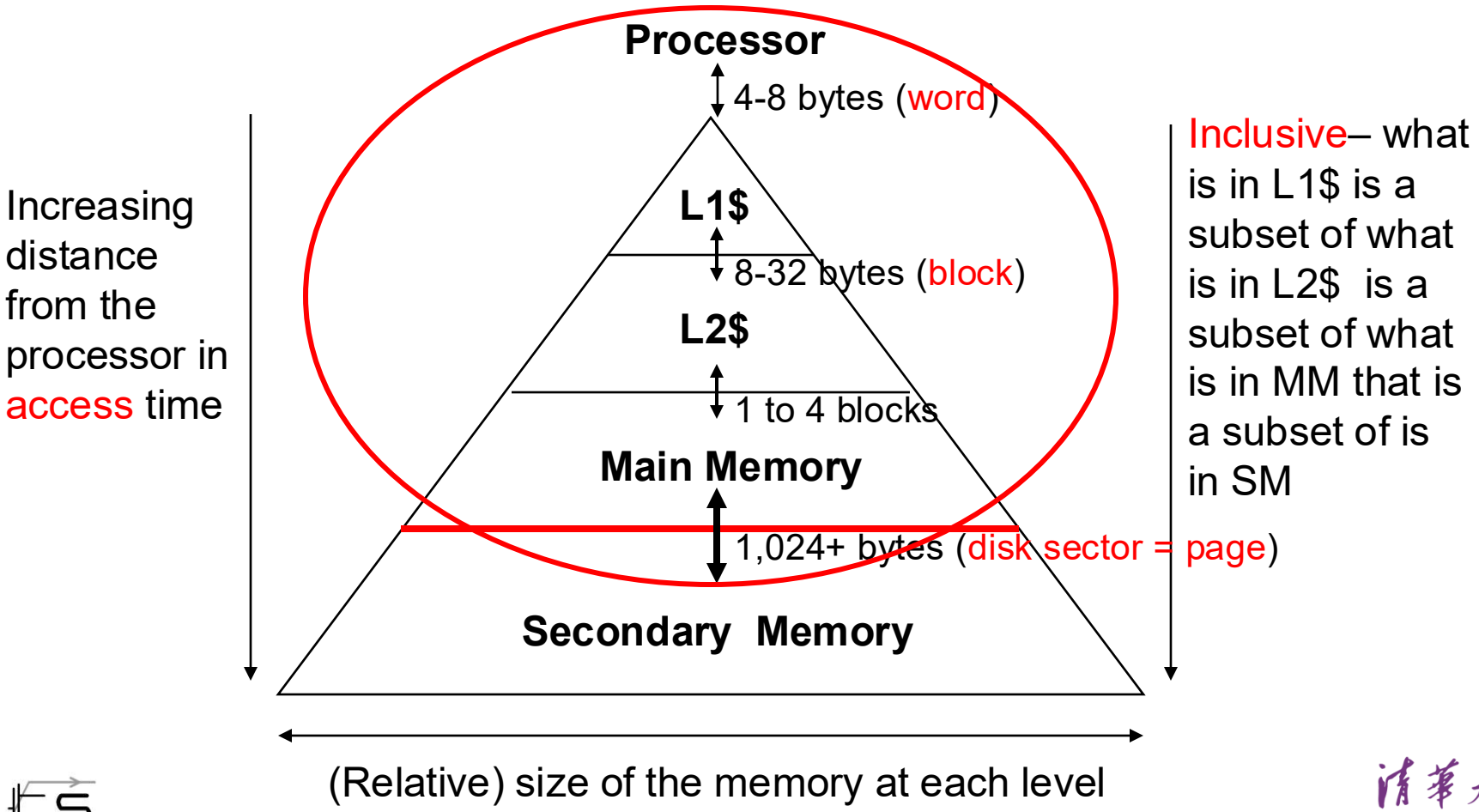
Yongpan Liu

ypliu@tsinghua.edu.cn

2025-Nov-19

Review: The Memory Hierarchy

- Take advantage of the principle of locality to present the user with as much memory as is available in the cheapest technology at the speed offered by the fastest technology

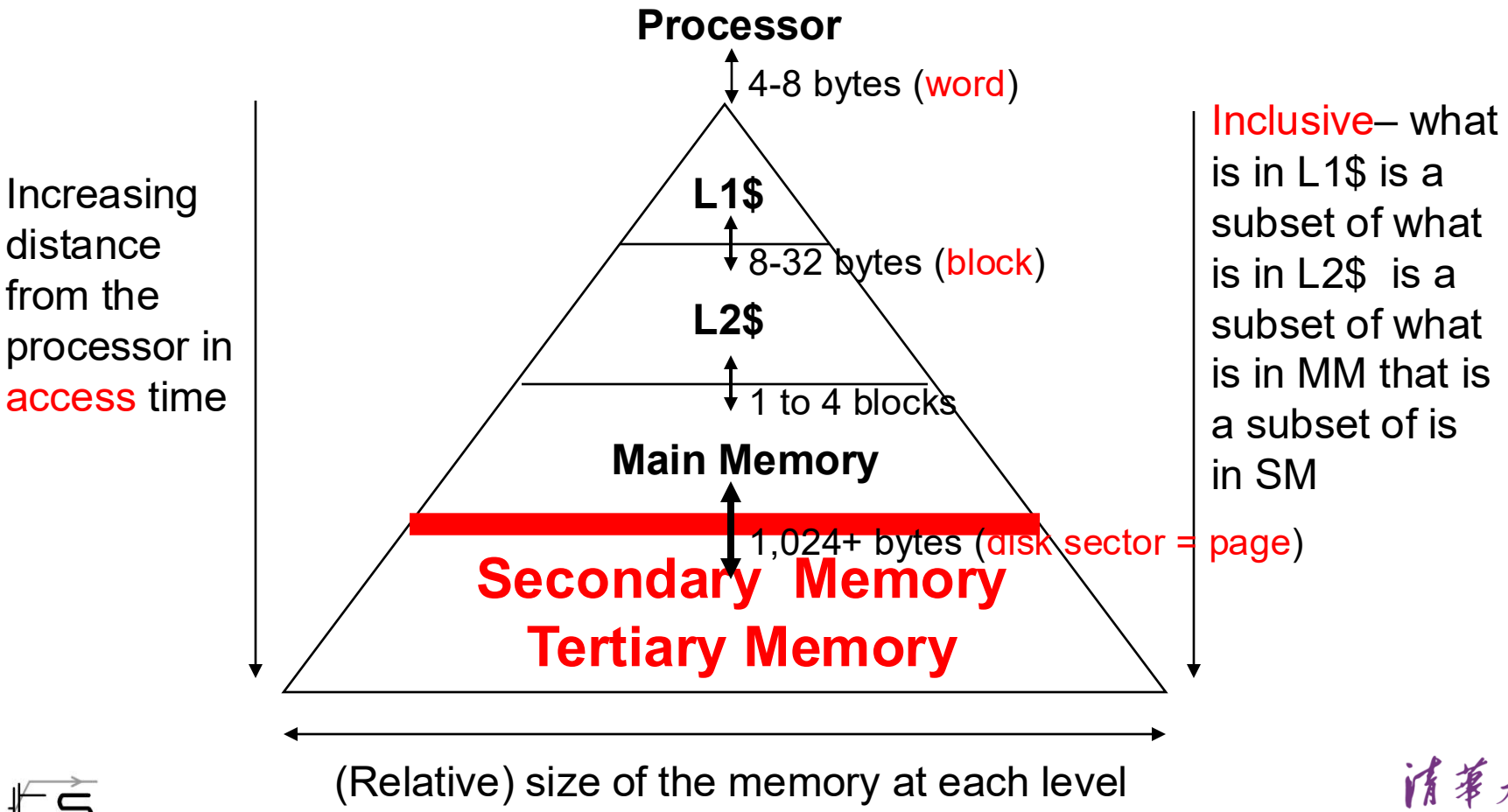


How is the Hierarchy Managed?

- registers ↔ memory
  - by compiler (programmer?)
- cache ↔ main memory
  - by the cache controller hardware
- main memory ↔ disks
  - by the operating system (virtual memory)
  - virtual to physical address mapping assisted by the hardware (TLB)
  - by the programmer (files)

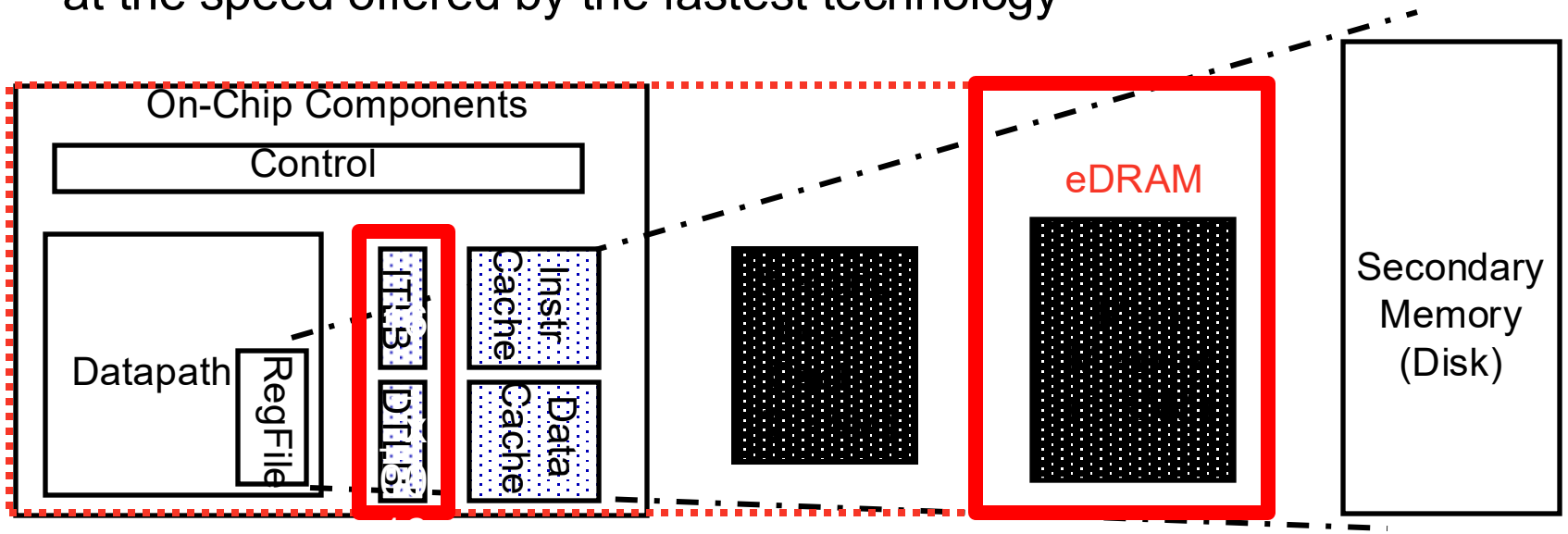
Review: The Memory Hierarchy

- Take advantage of the principle of locality to present the user with as much memory as is available in the cheapest technology at the speed offered by the fastest technology



A Typical Memory Hierarchy

- By taking advantage of the principle of locality
  - Can present the user with as much memory as is available in the cheapest technology
  - at the speed offered by the fastest technology



Speed (%cycles):	½'s	1's	10's	100's	1,000's
Size (bytes):	100's	K's	10K's	M's	G's to T's
Cost:	highest				lowest

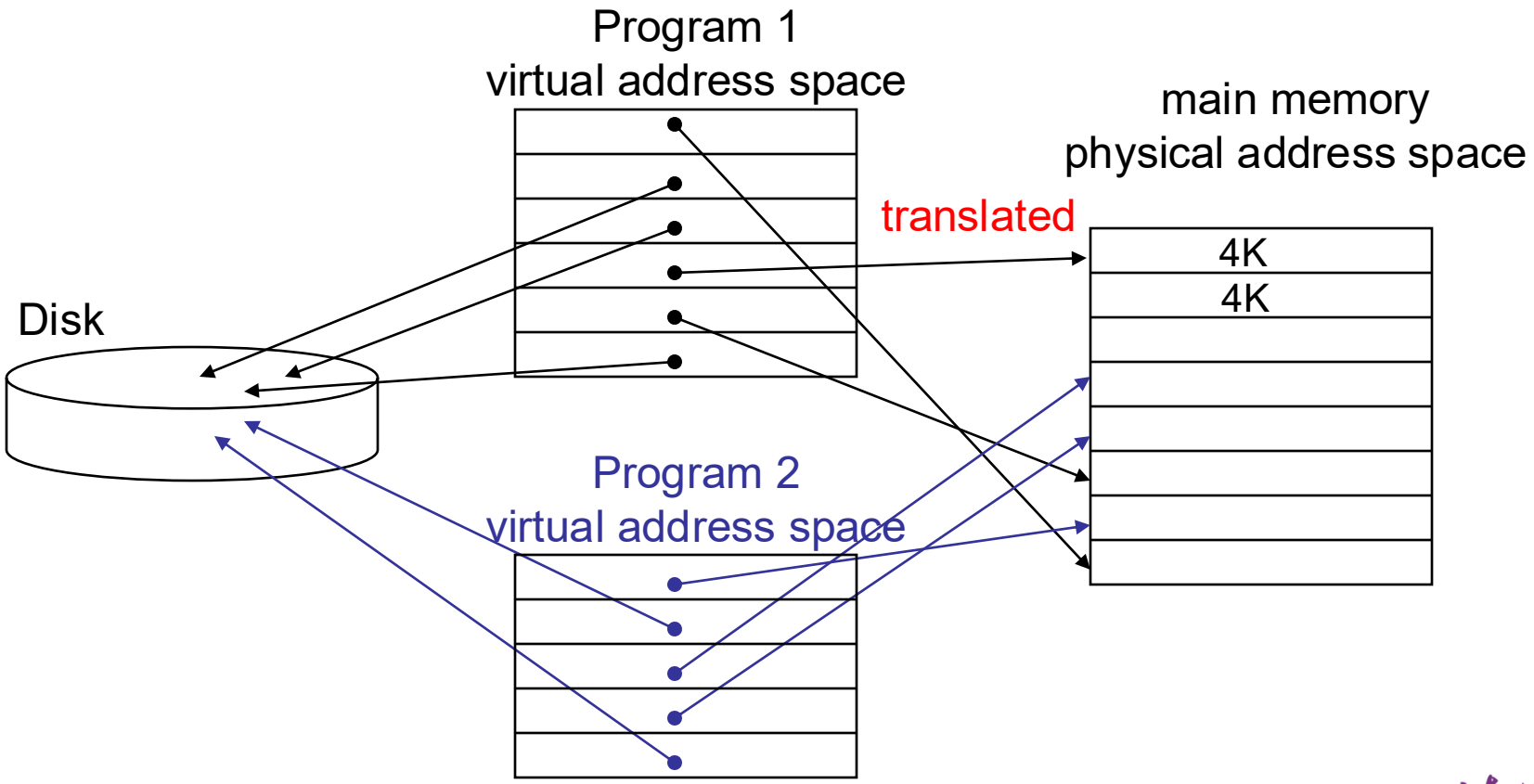
Translation Lookaside Buffer (TLB)

Virtual Memory

- Use Main Memory as a "cache" for secondary memory
  - Allows efficient and safe sharing of memory among multiple programs
  - Provides the ability to easily run programs larger than the size of physical main memory
  - Simplifies loading a program for execution by providing for code relocation (i.e., the code can be loaded anywhere in main memory)
- What makes it work? – again the Principle of Locality
  - A program is likely to access a relatively small portion of its address space during any period of time
- Each program is compiled into its own address space – a "virtual" address space
  - During run-time each virtual address must be translated to a physical address (an address in main memory)

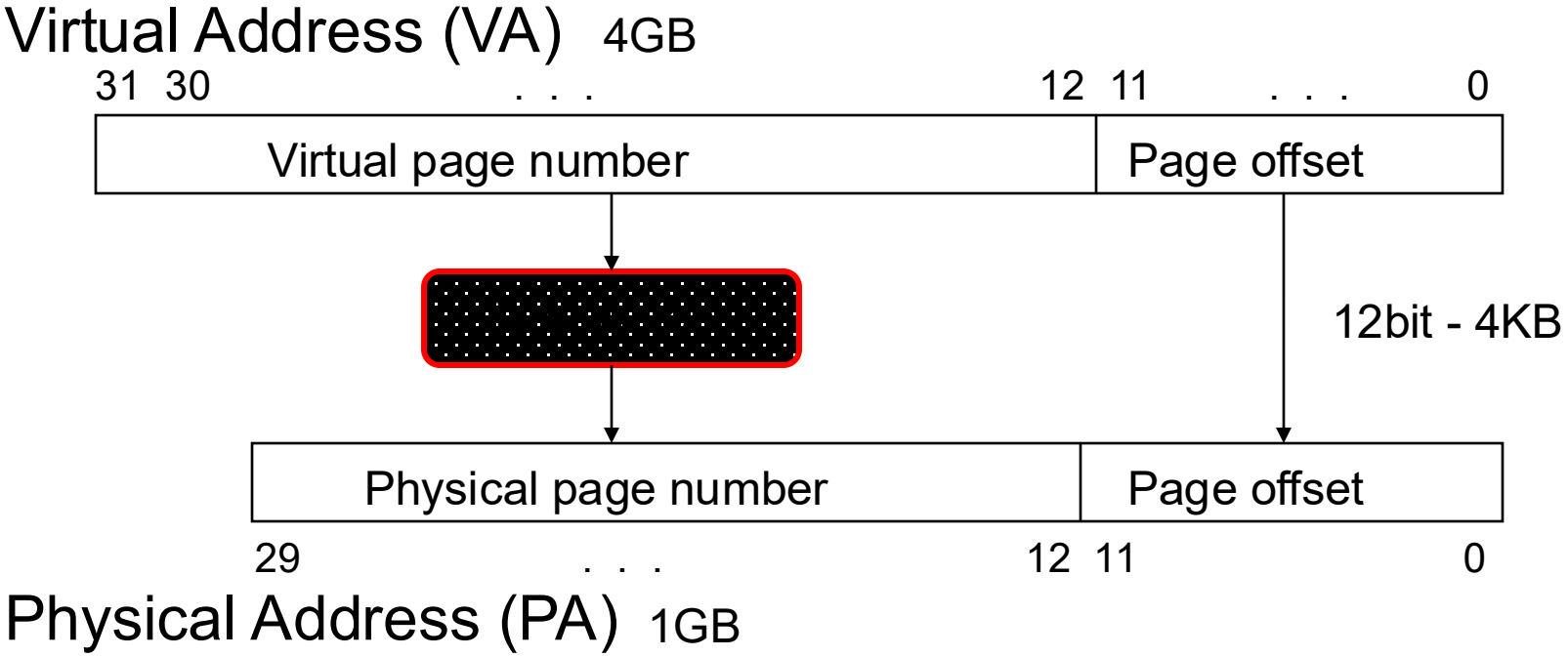
Two Programs Sharing Physical Memory

- A program's address space is divided into pages (all one fixed size) or segments (variable sizes)
  - The starting location of each page (either in main memory or in secondary memory) is contained in the program's page table



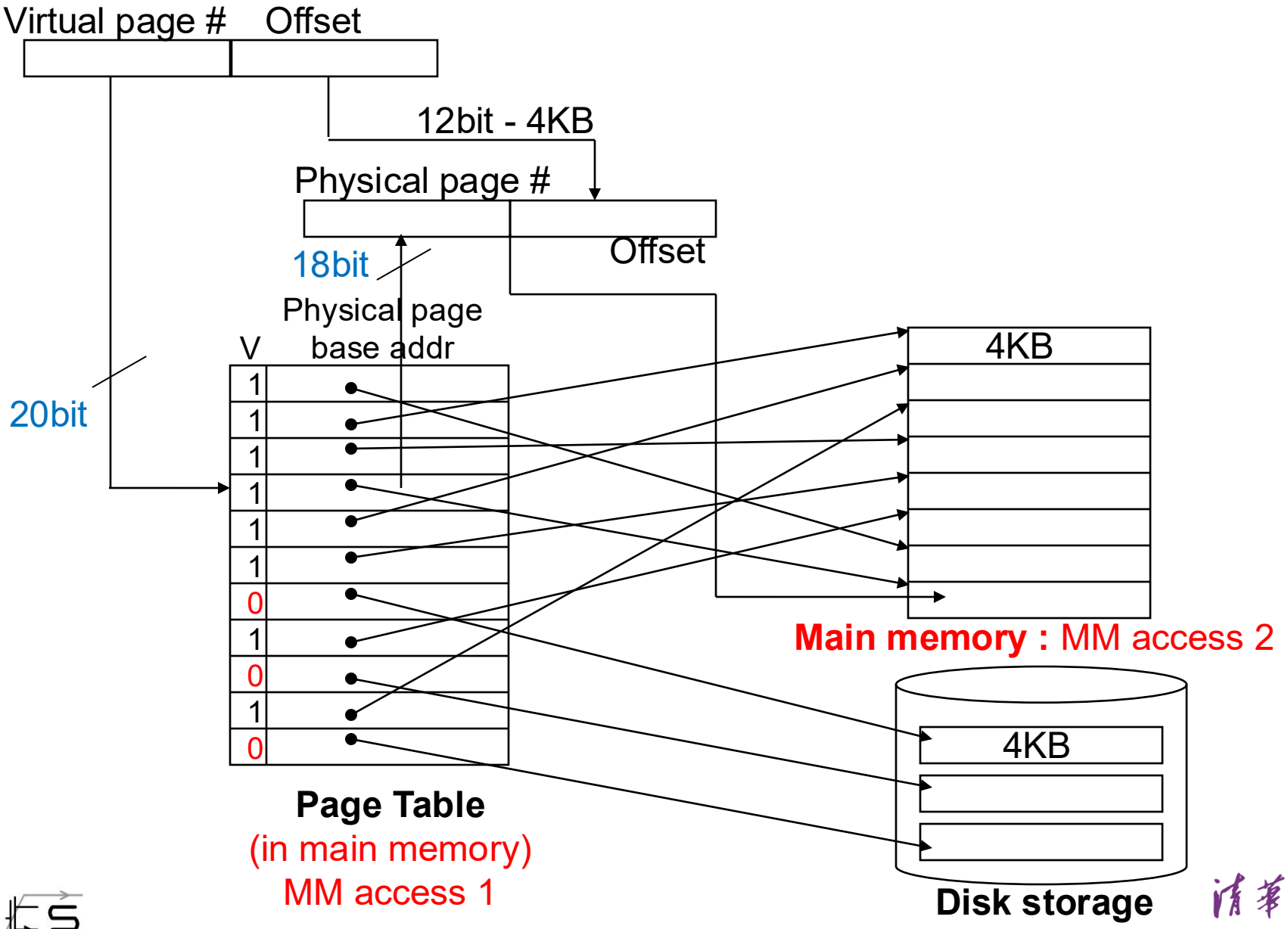
Address Translation

- A virtual address is translated to a physical address by a combination of hardware and software



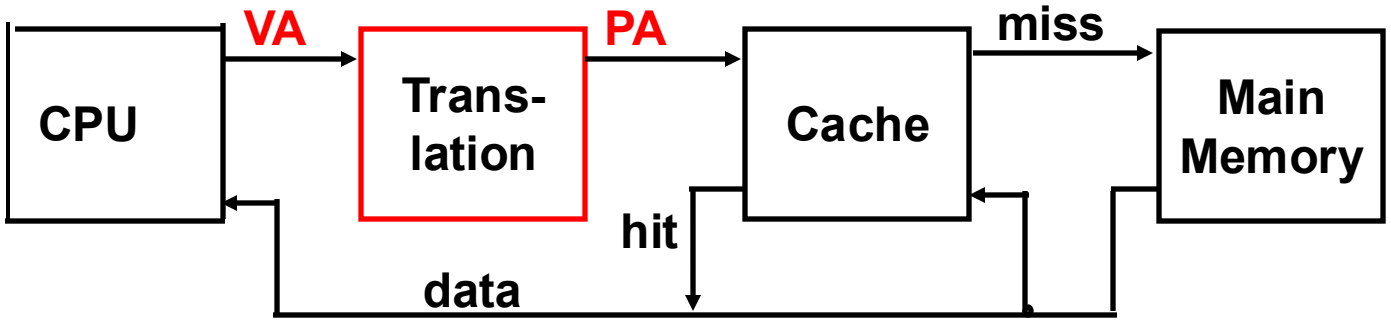
- So each memory request *first* requires an address translation from the virtual space to the physical space
  - A virtual memory miss (i.e., when the page is not in physical memory) is called a page fault (~ miss rate?)

Address Translation Mechanisms



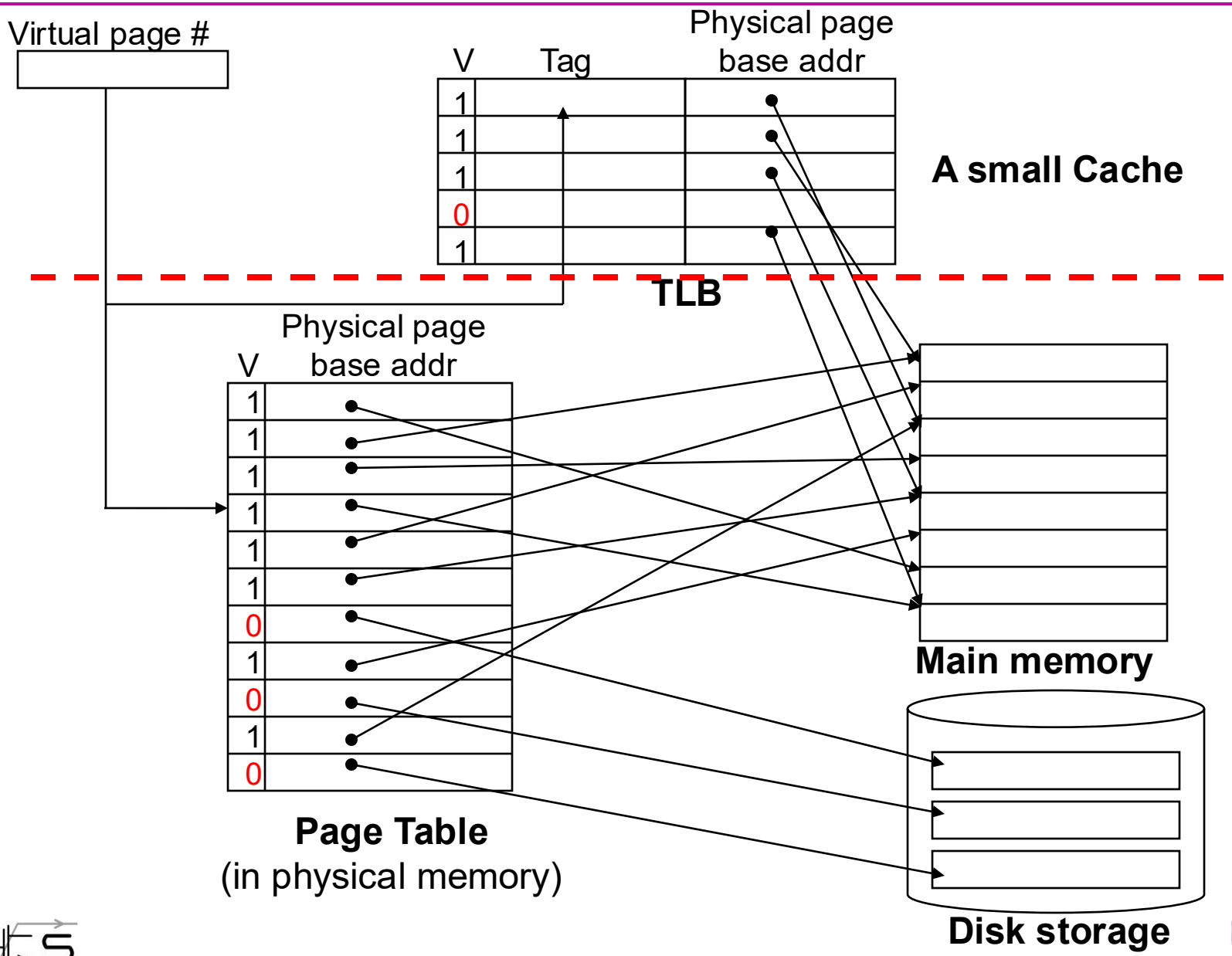
Virtual Addressing with a Cache

- Thus it takes an *extra* memory access to translate a VA to a PA



- This makes memory (cache) accesses very expensive (if every access was really *two* accesses to MM)
- The hardware fix is to use a Translation Lookaside Buffer (TLB) – a small cache that keeps track of recently used address mappings to avoid having to do a page table lookup (access locality of page table: Time / Space)

Making Address Translation Fast



Translation Lookaside Buffers (TLBs)

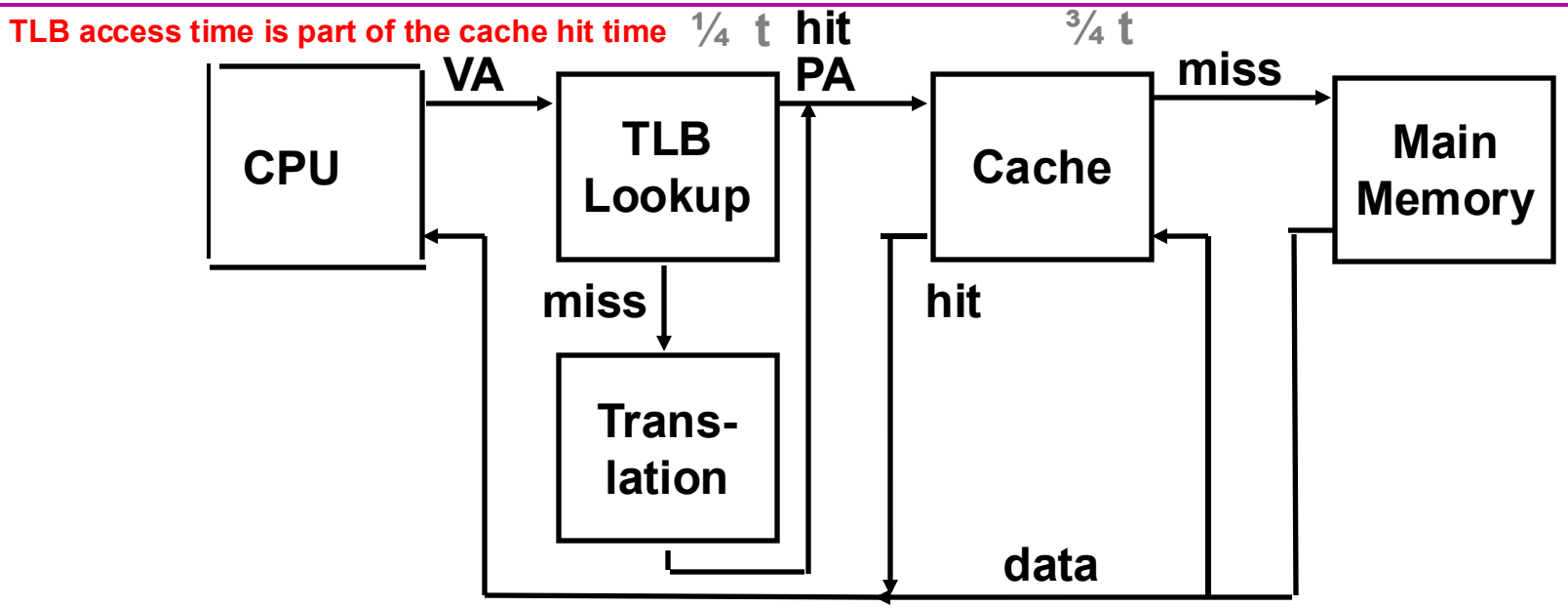
- Just like any other cache, the TLB can be organized as fully associative, set associative, or direct mapped

V	Virtual Page #	Physical Page #	Dirty	Ref

One entry

- TLB access time is typically smaller than cache access time (because TLBs are much smaller than caches)
  - TLBs are typically not more than 128 to 256 entries even on high end machines

A TLB in the Memory Hierarchy



- A TLB miss – is it a page fault or merely a TLB miss?
  - If the page is loaded into main memory, then the TLB miss can be handled (in hardware or software) by loading the translation information from the page table into the TLB
    - Takes 10's of cycles to find and load the translation info into the TLB
  - If the page is not in main memory, then it's a true page fault
    - Takes 1,000,000's of cycles to service a page fault
- TLB misses are much more frequent than true page faults

Some Virtual Memory Design Parameters

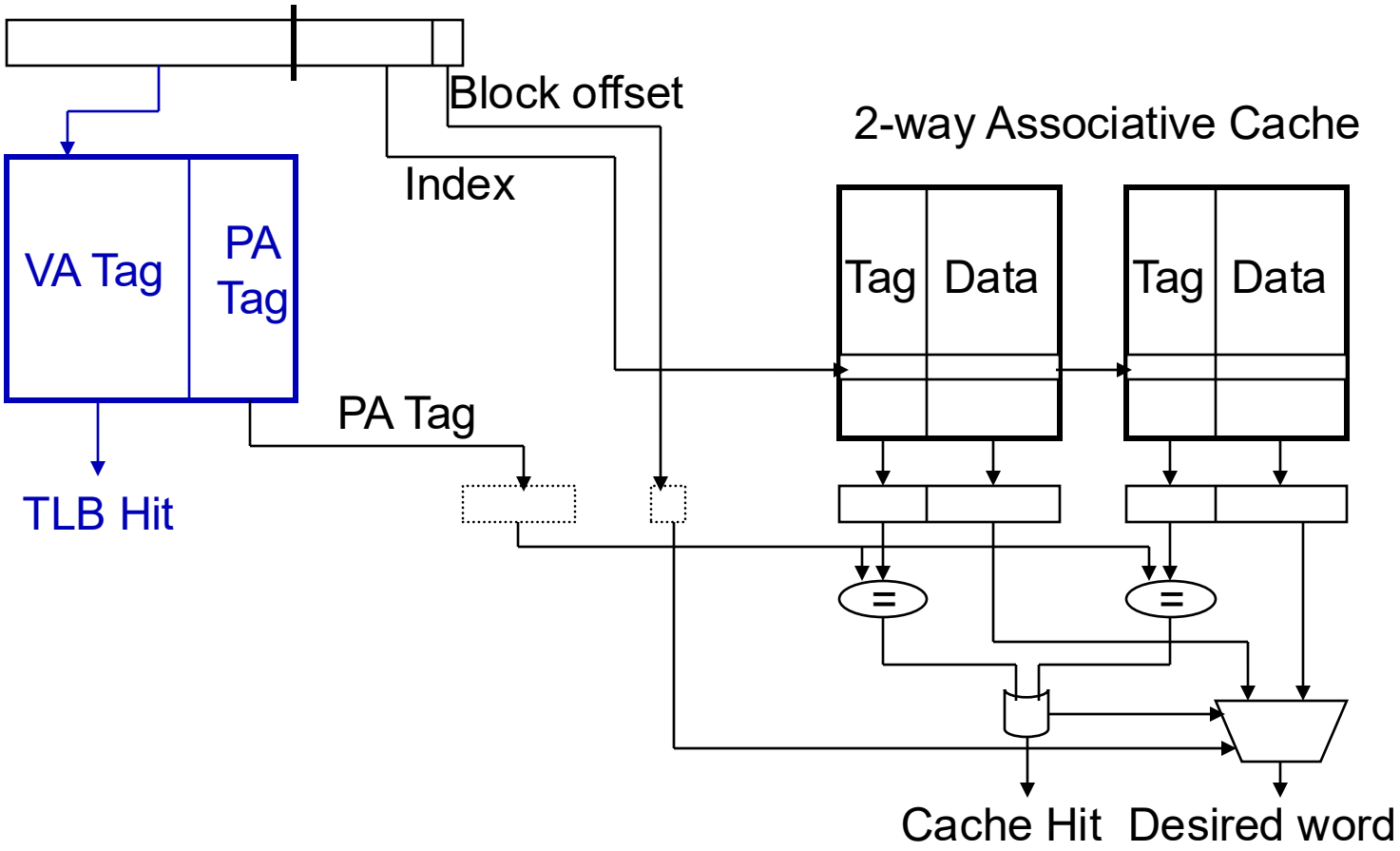
	Paged VM	TLBs
Total size	16,000 to 250,000 words	16 to 512 entries
Total size (KB)	250,000 to 1,000,000,000	0.25 to 16
Block size (B)	4000 to 64,000	4 to 32
Miss penalty (clocks)	10,000,000 to 100,000,000	10 to 1000
Miss rates Page fault	0.00001% to 0.0001%	0.01% to 2%

Two Machines' Cache Parameters

	Intel P4	AMD Opteron
TLB organization	1 TLB for instructions and 1 TLB for data Both 4-way set associative Both use ~LRU replacement Both have 128 entries TLB misses handled in hardware	2 TLBs for instructions and 2 TLBs for data Both L1 TLBs fully associative with ~LRU replacement Both L2 TLBs are 4-way set associative with round-robin Both L1 TLBs have 40 entries Both L2 TLBs have 512 entries TLB misses handled in hardware

Integrating VM, TLBs and Caches

- Can overlap the cache access with the TLB access
  - Works when the high order bits of the VA are used to access the TLB while the low order bits are used as index into cache



TLB Event Combinations

TLB	Page Table	Cache	Possible? Under what circumstances?
Hit	Hit	Hit	Yes – what we want!
Hit	Hit	Miss	Yes – although the page table is not checked if the TLB hits
Miss	Hit	Hit	Yes – TLB miss, PA in page table
Miss	Hit	Miss	Yes – TLB miss, PA in page table, but data not in cache
Miss	Miss	Miss	Yes – page fault
Hit	Miss	Miss/ Hit	**Impossible** – TLB translation not possible if page is not present in memory
Miss	Miss	Hit	**Impossible** – data not allowed in cache if page is not in memory

The Hardware/Software Boundary

- What parts of the virtual to physical address translation is done by or assisted by the hardware?
  - Translation Lookaside Buffer (TLB) that caches the recent translations
    - TLB access time is part of the cache hit time
    - May allot an extra stage in the pipeline for TLB access
  - Page table storage, fault detection and updating
    - Page faults result in interrupts (precise) that are then handled by the OS
    - Hardware must support (i.e., update appropriately) Dirty and Reference bits (e.g., ~LRU) in the Page Tables
  - Disk placement
    - Bootstrap (e.g., out of disk sector 0) so the system can service a limited number of page faults before the OS is even loaded

Summary

- The Principle of Locality:
  - Program likely to access a relatively small portion of the address space at any instant of time.
    - Temporal Locality: Locality in Time
    - Spatial Locality: Locality in Space
- Caches, TLBs, Virtual Memory all understood by examining how they deal with the four questions
  1. Where can block be placed?
  2. How is block found?
  3. What block is replaced on miss?
  4. How are writes handled?
- Page tables map virtual address to physical address
  - TLBs are important for fast translation

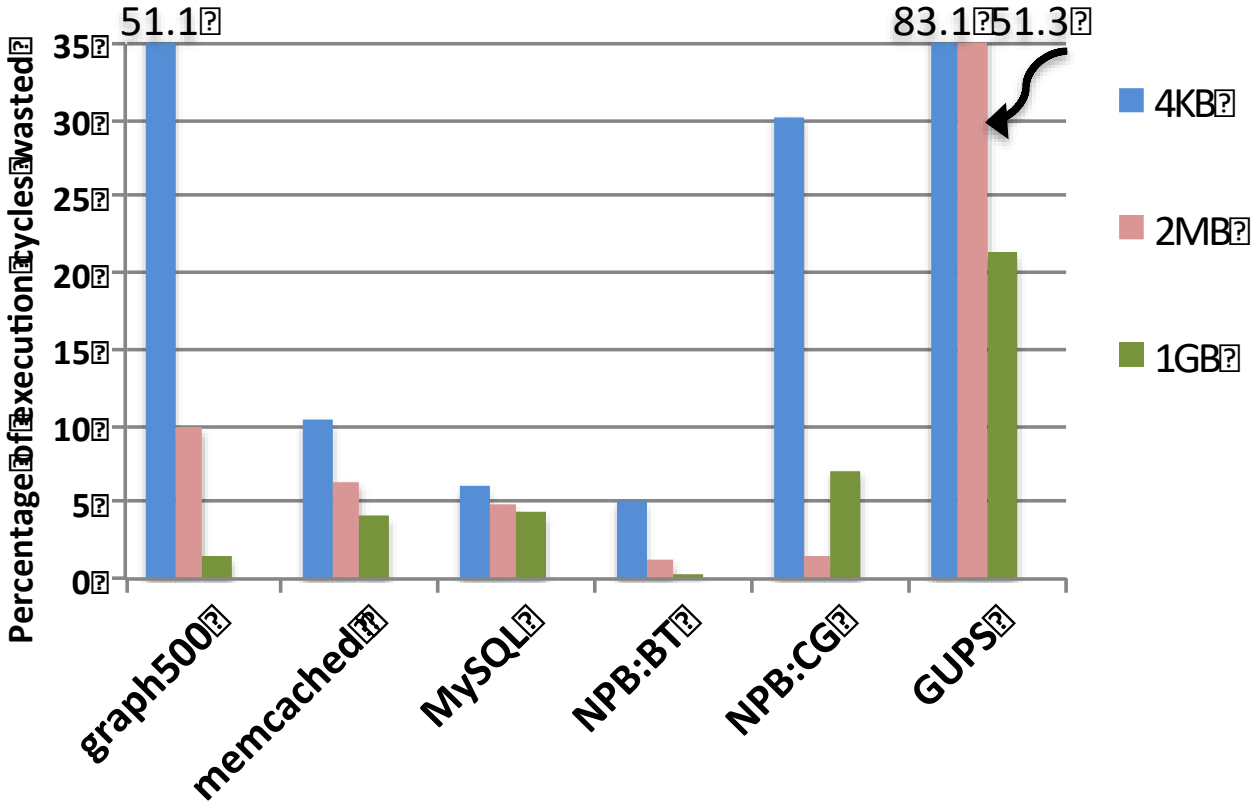
Recent Research on Virtual Memory

- Arkaprava Basu, Jayneel Gandhi, Jichuan Chang, Mark D. Hill, and Michael M. Swift. 2013. Efficient virtual memory for big memory servers. (ISCA '13).

- Rachata Ausavarungnirun, Joshua Landgraf, Vance Miller, Saugata Ghose, Jayneel Gandhi, Christopher J. Rossbach, and Onur Mutlu. 2017. Mosaic: a GPU memory manager with application-transparent support for multiple page sizes. (MICRO-50 '17).

- Abhishek Bhattacharjee. 2017. Translation-Triggered Prefetching. (ASPLOS '17).

ISCA ’13: “Big-Memory” Sever Workloads



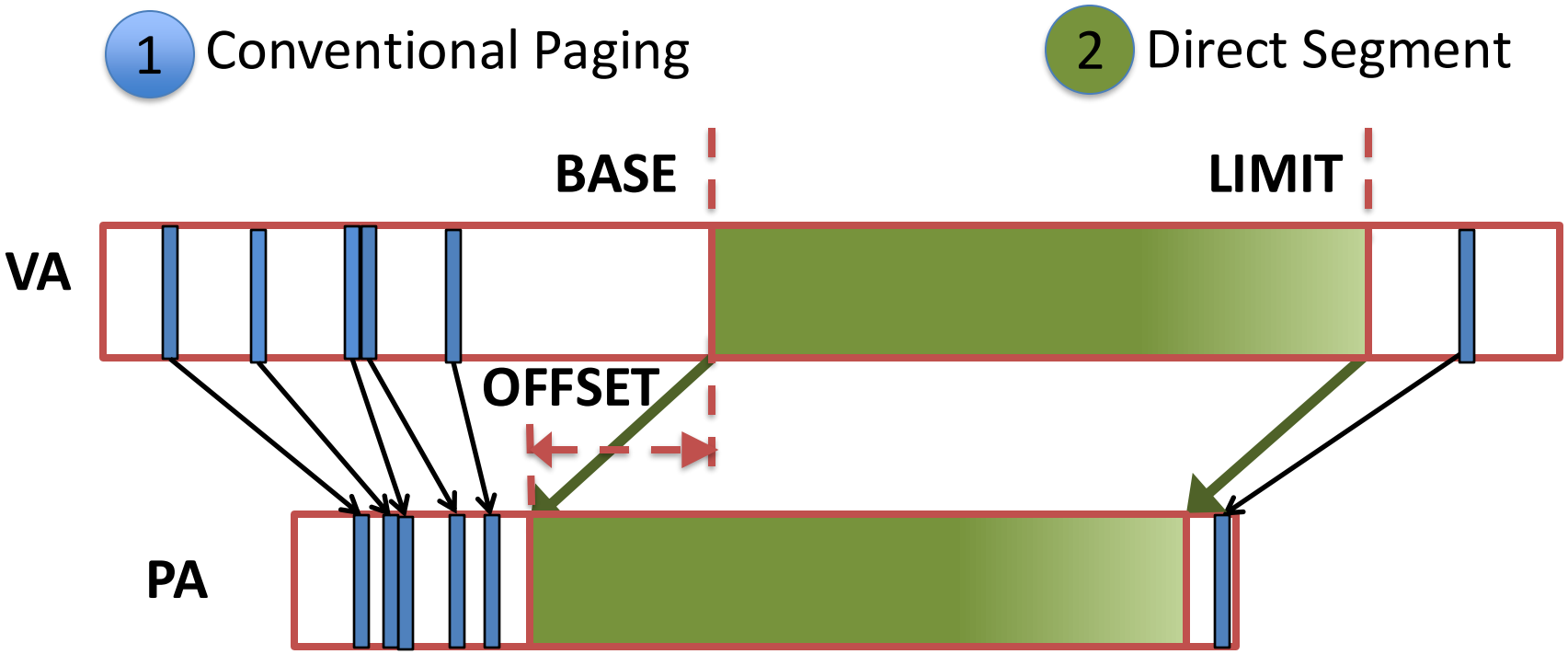
- 常见的服务器应用（如数据库等）中，TLB Miss可造成巨大的性能损失；
- 使用大 Page Size 不能完美解决该问题

Actual Use of Virtual Memory

- **Swapping**: big-memory workloads do little or no swapping, because performance-critical applications cannot afford to wait for disk I/Os.
- **Memory allocation and fragmentation**: big-memory workloads rarely suffer from OS-visible fragmentation, as they allocate most memory during startup and then manage that memory internally.
- **Per-page permissions**: nearly all of the memory in big-memory workloads is dynamically allocated memory with read-write permission

Most features brought by virtual memory are not used!

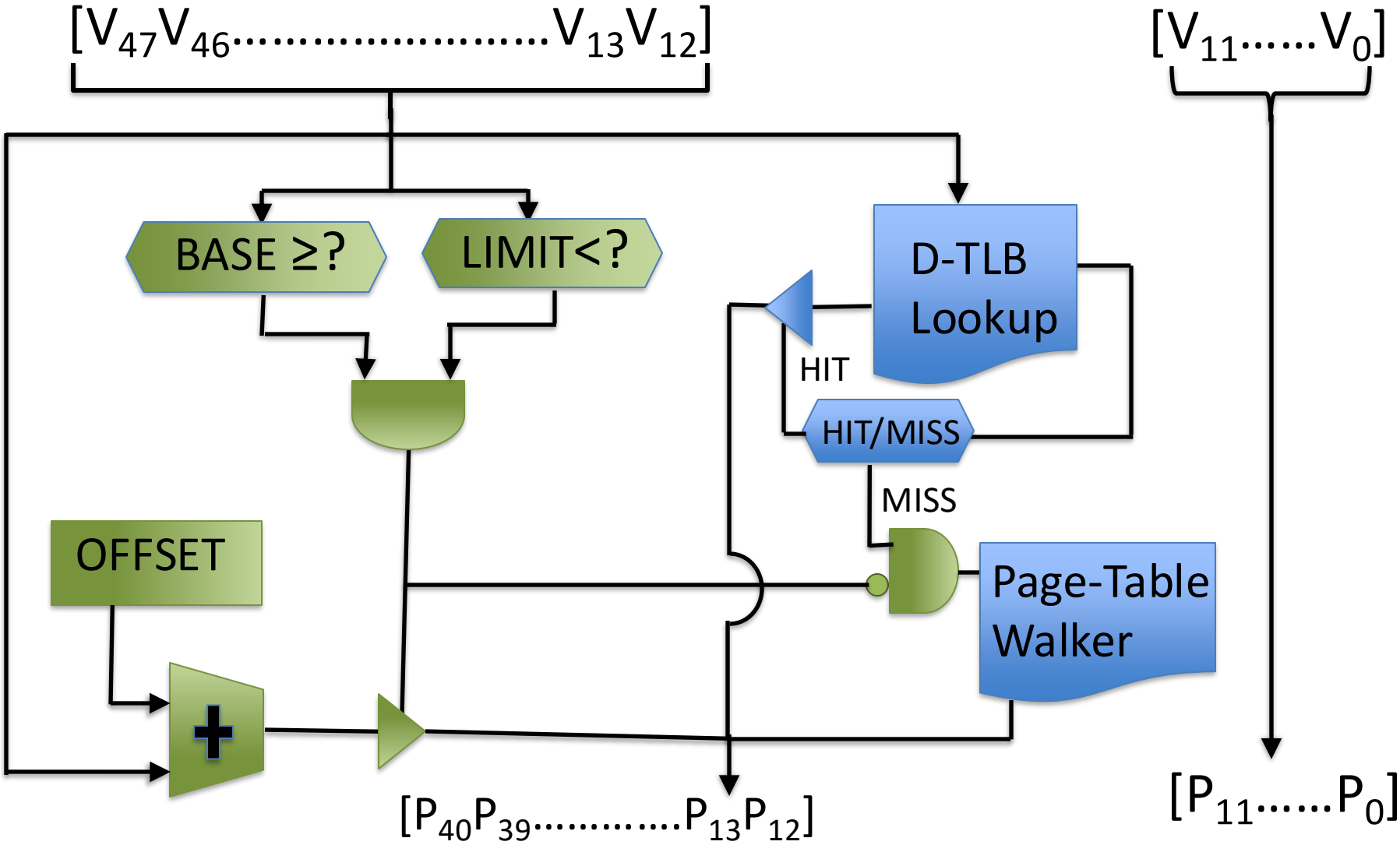
Introducing Direct Segment



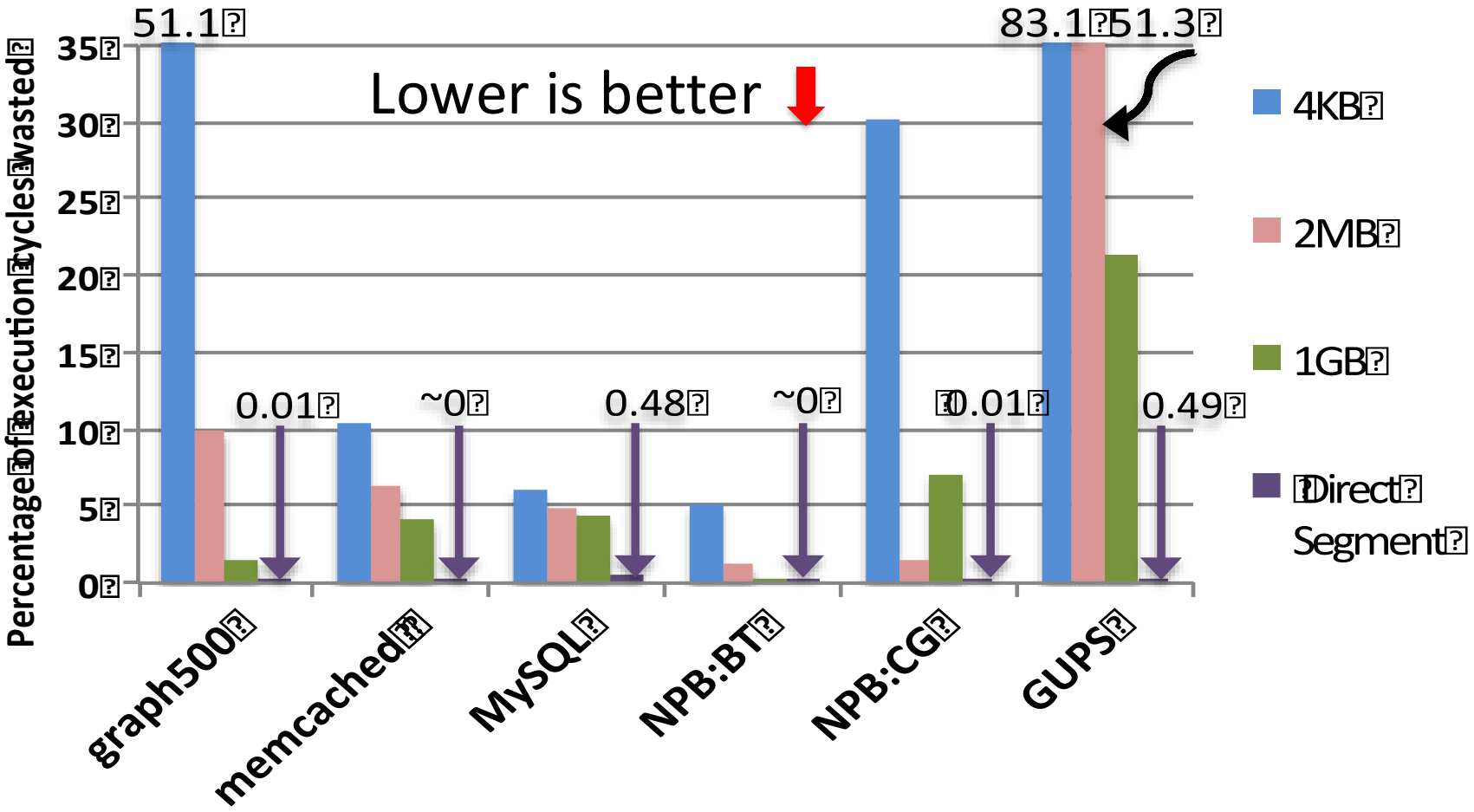
Why Direct Segment?

- Matches big memory workload needs
- NO TLB lookups => NO TLB Misses

Hardware Support for Direct Segment

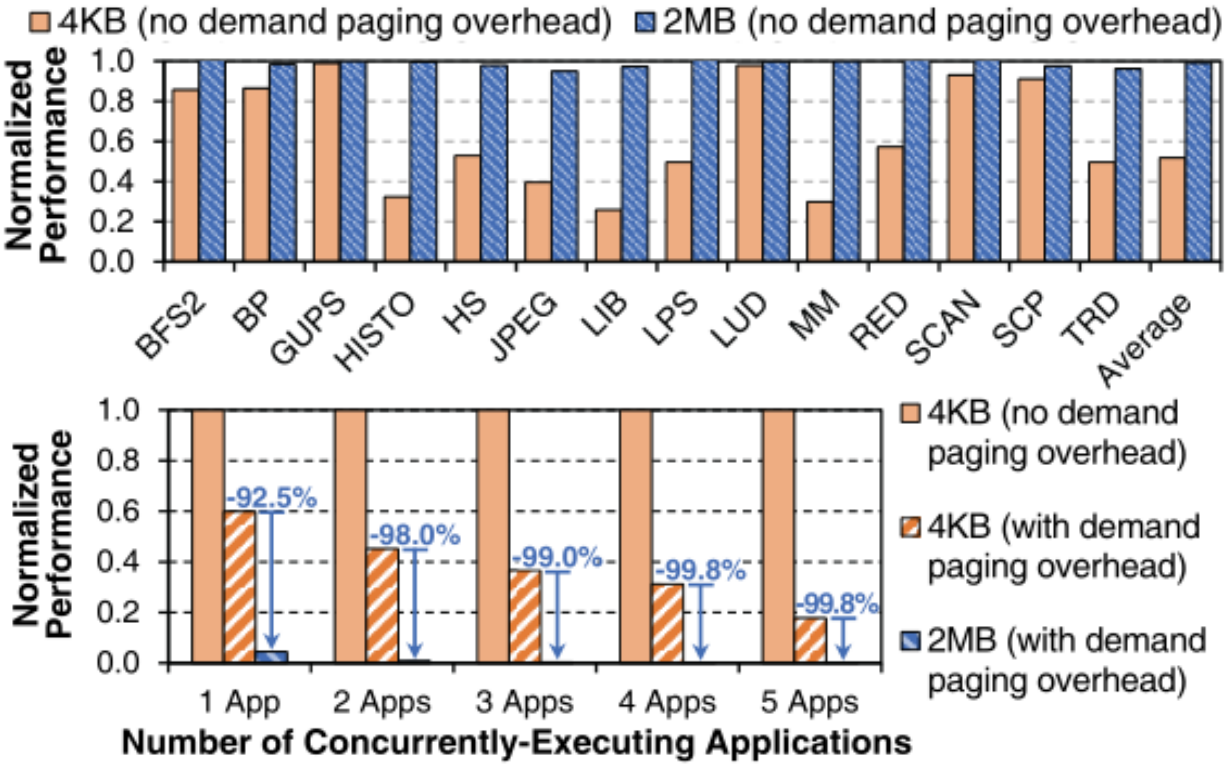


Evaluation Result



Tsinghua University

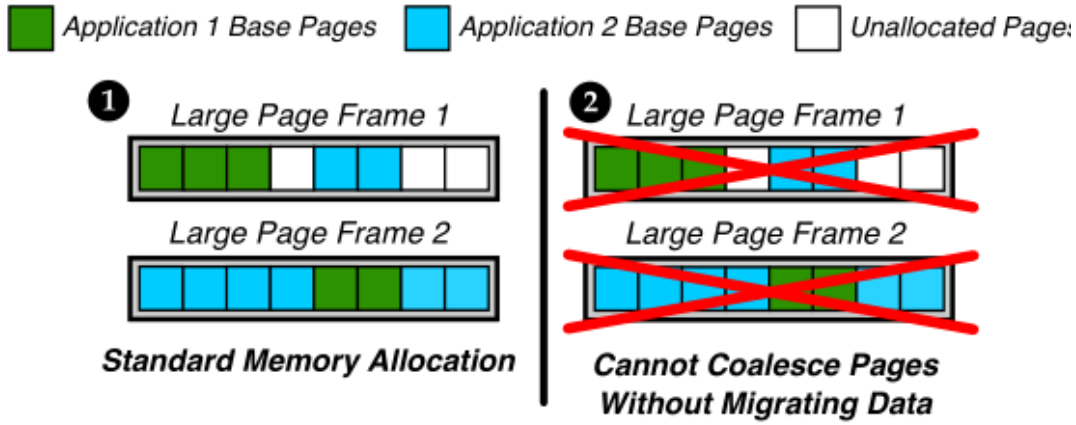
MICRO-50 ’17: VM Challenge for GPU



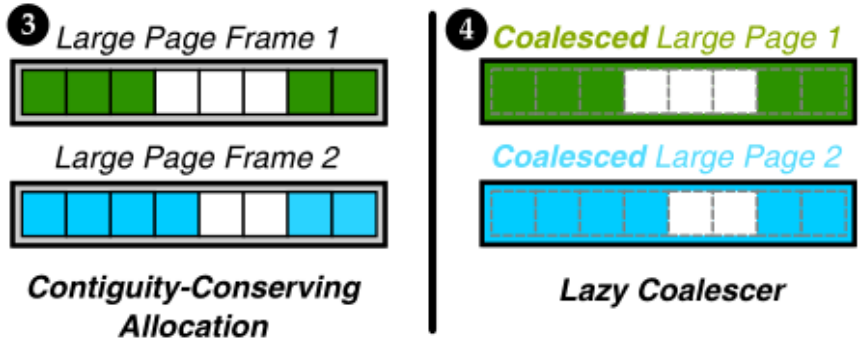
- A single TLB miss stall hundreds of threads at once
- Larger page size brings less TLB miss, but leads to memory bloat and longer memory access latencies.

Multiple Page Sizes: Coalesce Base Pages into Large Page

- Using large pages for address translation, while providing better demand paging performance by using base pages for data transfer.

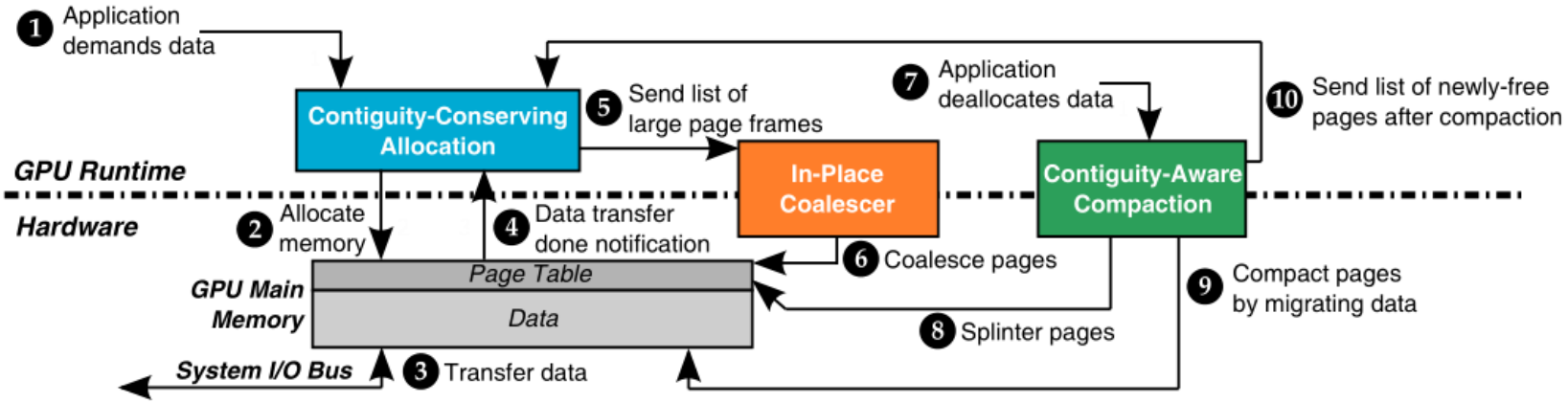


(a) State-of-the-art GPU memory management [92].



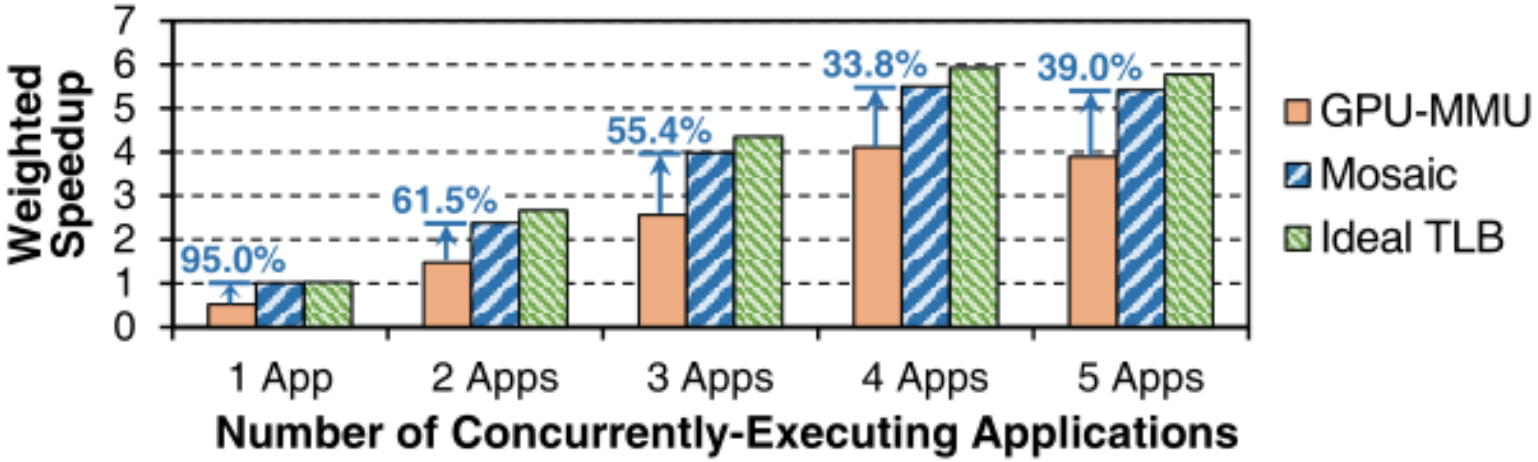
(b) Memory management with *Mosaic*.

System Overview

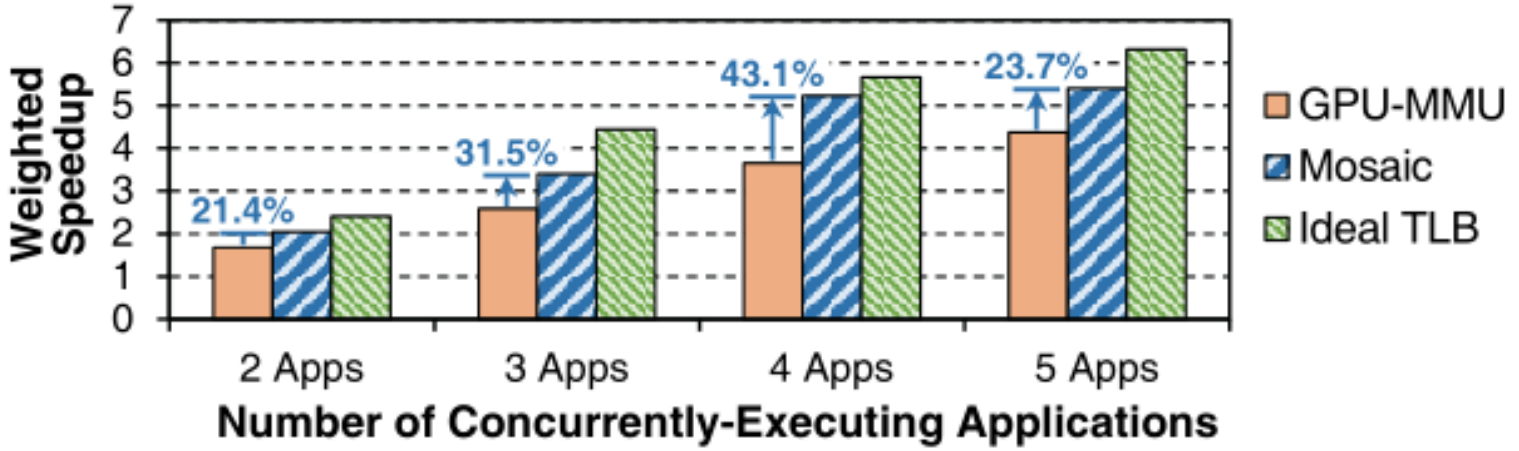


- **Contiguity-Conserving Allocation**: conserve the contiguity of base pages & guarantee that a single large page frame contains base pages from only a single application;
- **In-Place Coalescer**: check and coalesce contiguous base pages;
- **Contiguity-Aware Compaction**: splinter large page with a high degree of internal fragmentation into base pages.

Evaluation Result



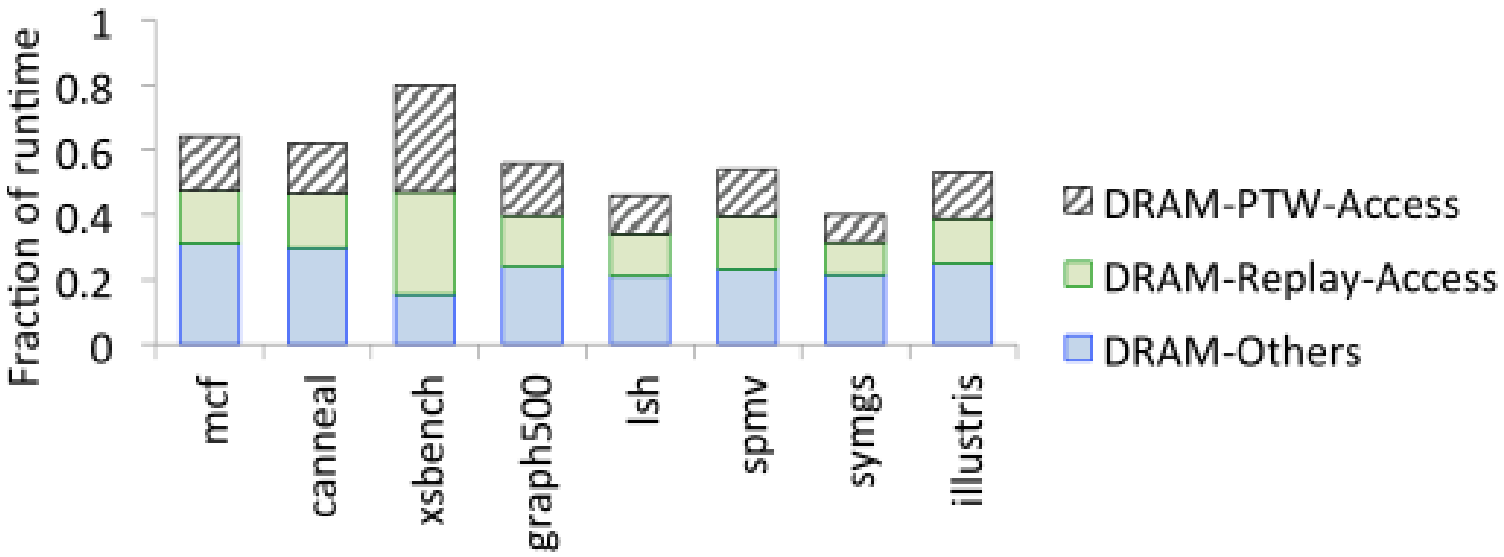
Homogeneous workload



Heterogeneous workload

Tsinghua University

ASPLOS ’17: DRAM Replayed Accesses



- **DRAM Replayed Accesses**: Memory access after a TLB miss and page table access.
- DRAM accesses for replayed data (in green) constitute 10-30% of runtime
- TLB miss indicates cold data, which usually follows by a cache miss.

Prefetch Opportunity

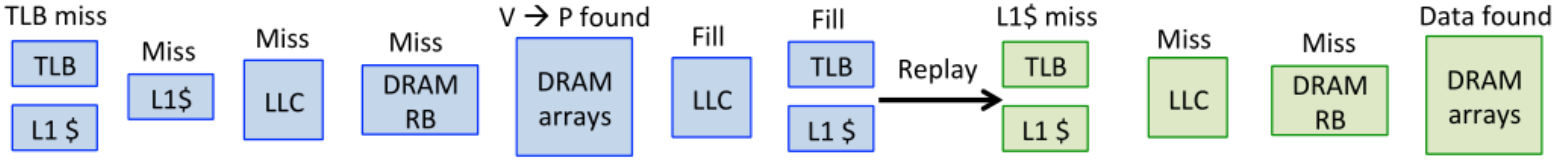


Figure 5. Timeline of events for a memory reference that misses in the TLB. Page table walks are shown in blue the memory replay is shown in green.

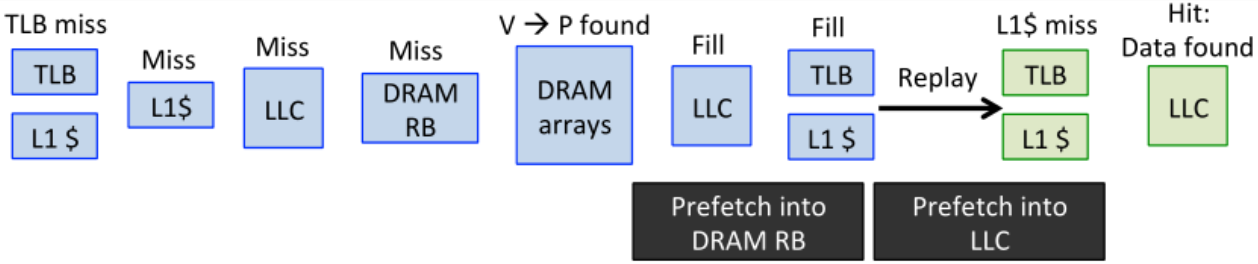
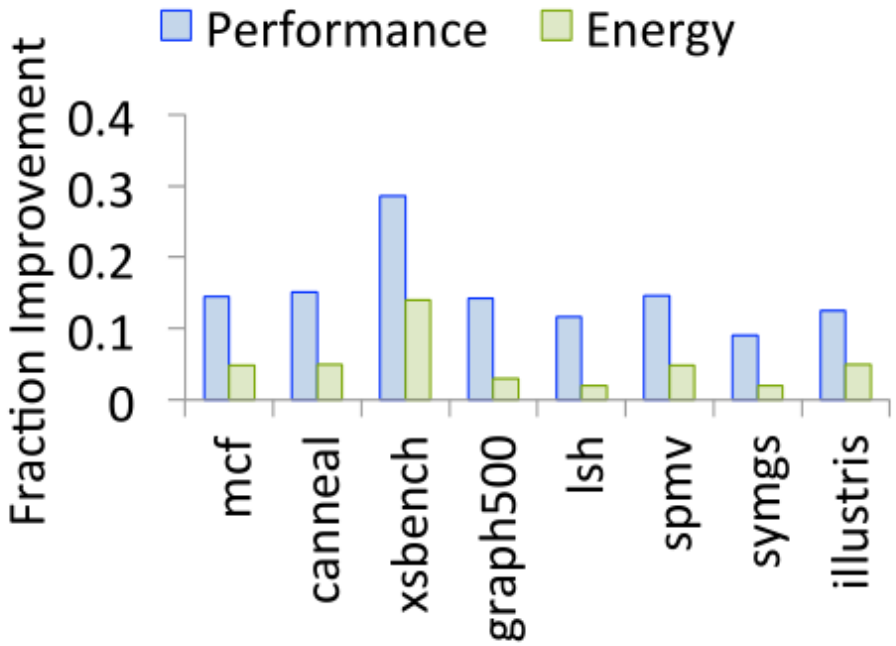


Figure 6. Timeline of events when TEMPO prefetches the data that the replayed instruction will use into the DRAM row buffer and LLC. Subsequent LLC and row buffer hits improve performance.

- 120+ cycles (Intel Haswell/Skylake) between page table walk and replayed access LLC(Last Level Cache) lookup.
- Prefetch data into DRAM row buffer and LLC.

Evaluation Result



- Up to 30% performance boosts and 14% energy savings.